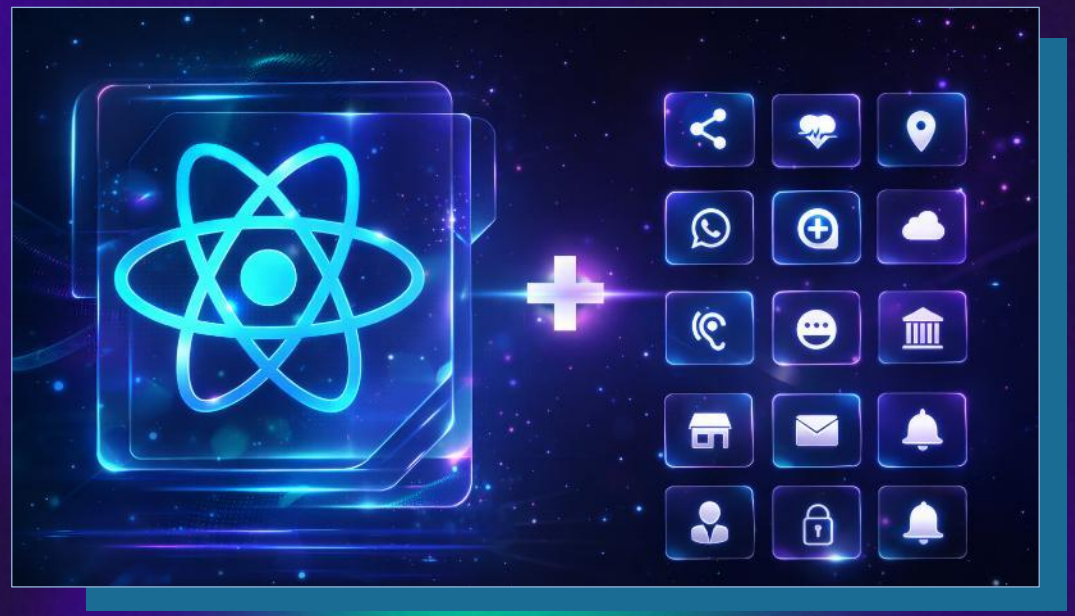
React Apps

React-Router, Pages and Navigation, Tailwind, Supabase Integration



Svetlin Nakov, PhD

Co-founder @ SoftUni

SoftUni AI

https://ai.softuni.bg

Agenda

SoftUni AI

1. **Fetching Data from an API**: useEffect() and HTTP fetch()
2. **TanStack Query**: simplified API requests for React components
3. **React Router Basics**: configuring routes, page navigation
4. **Dynamic Routing and Layouts**: dynamic URL parameters
5. **Supabase with React**: using Supabase from React, Supabase MCP, authentication, CRUD operations
6. **Intro to Tailwind**: the "utility-first" concept, basic Tailwind classes, examples
7. **React + Tailwind**: React apps with Vite, TypeScript, Tailwind, and React Router, examples
8. **Building a Blog** with **React + Tailwind + Supabase**

Sli.do Code

AppsAI

Join at

slido.com

#AppsAI

Breaks

20:00 / 21:00

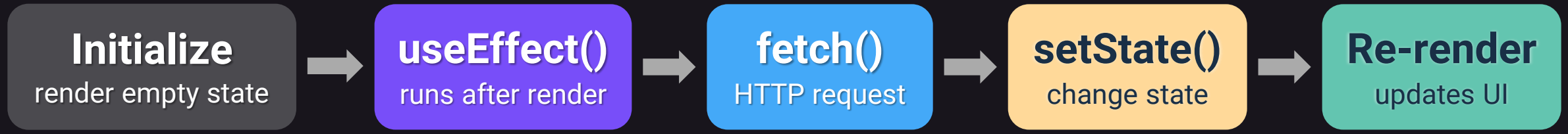
Fetching Data from an API

Loading Data after Rendering in React:
useEffect() → fetch() → set state → re-render

Fetching Data in React

SoftUni AI

- **Loading external data** from React component:



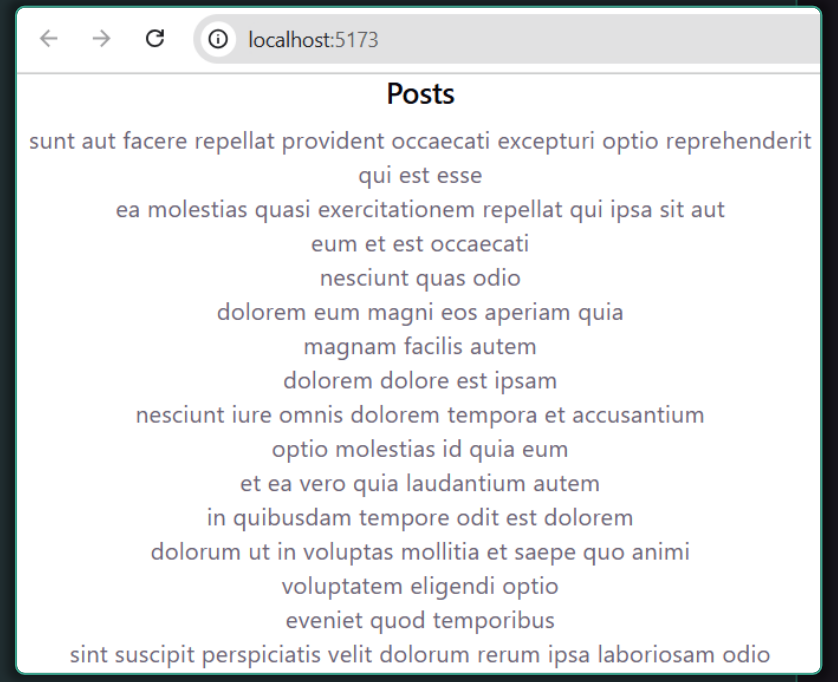
1. Set **initial state** to an **empty** array or null
2. Load data inside **useEffect()** after mount (use **fetch()**)
3. Call **setState()** when data arrives → triggers re-render

- **useEffect()** hook runs code **after** the component renders
  - Used for **data loading**, timers, and other side effects
  - The **dependency array [ ]** means "run once on mount"

Fetching Data in React – Example

SoftUni AI

```
function Posts() {
  const [posts, setPosts] = useState([]);
  useEffect(() => {
    fetch("https://jsonplaceholder.typicode.com/posts")
      .then((response) => response.json())
      .then((json) => setPosts(json));
  }, []); // runs once after component in rendered
  return (
    <div>
      <h2>Posts</h2>
      {posts.map((item) => (<p key={item.id}>{item.title}</p>))}
    </div>
  );
}
```



The HTTP fetch() API Function

SoftUni AI

- **fetch()** is a built-in function for HTTP requests
  - Basic HTTP methods: **GET**, **POST**, **PUT**, **DELETE**
  - Returns a **Promise** — resolved when the response arrives
  - Convert response to JSON with **response.json()**
  - Use **async** / **await** for cleaner, readable code
- Handle all response states for better UX:
  - **Loading** — show a spinner while the request runs
  - **Error** — show a friendly message if the request fails
  - **Empty** — tell the user when there is no data to show

Example: Fetching and Rendering

- Load users from the **JSONPlaceholder** test API (HTTP GET)
  - Endpoint: https://jsonplaceholder.typicode.com/users
  - Keep users in the **component state** with **useState()**
  - Render users in a **table or cards** layout
  - Show **loading** and **error** states while fetching
- AI prompt:

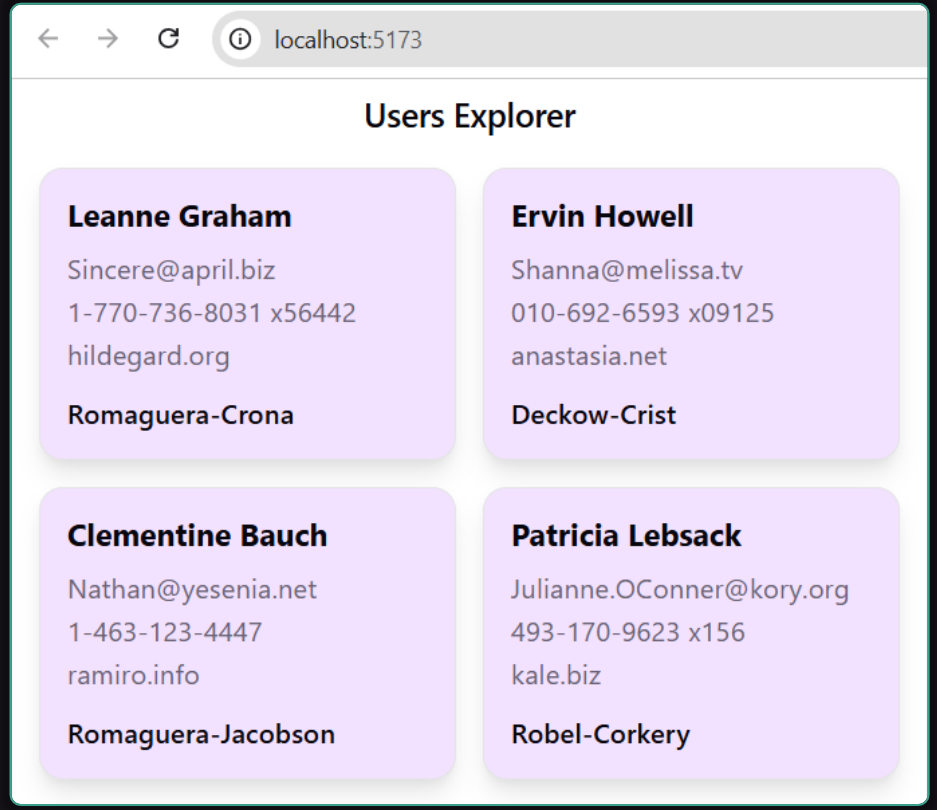
> Create a React component `UsersExplorer` to fetch and render users from https://jsonplaceholder.typicode.com/users. Handle loading / error / empty states.

Example: Fetching and Rendering (2)

SoftUni AI

```tsx
function UsersExplorer() {
  const [users, setUsers] = useState<User[]>([]);
  const [isLoading, setIsLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {···
  }, []);
```

```tsx
if (isLoading) return <p>Loading users...</p>;

if (error) return <p>{error}</p>;

if (users.length === 0)
  return <p>No users found.</p>;

return (
  <section className="users-explorer">···
  </section>
);
```



Example: Creating a New User

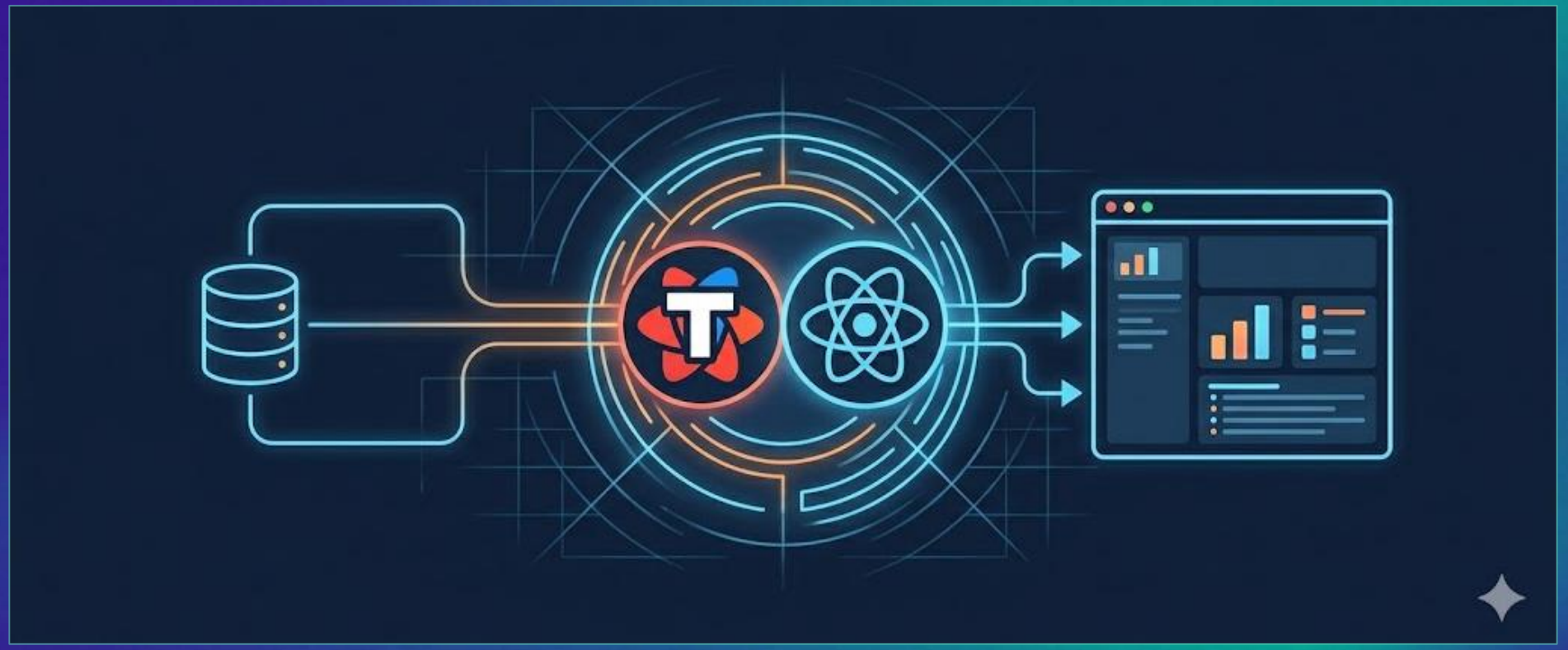
SoftUni AI

- Build a small form: **name, email, phone**
  - Submit with **fetch(..., { method: "POST" })**
  - Include **Content-Type: application/json** header
  - **JSONPlaceholder** is **demo-only** API → it simulates POST but does not really save posted data
  - After a successful POST, **append the new user** to the UI state
- AI prompt:

> Extend the `UsersExplorer` component: fetch users (GET), add a create-user form (POST), and append the created user to the list

TanStack Query

Simplified Access to APIs from React



TanStack Query

- **TanStack Query** – popular React library for **fetching**, **caching**, and **synchronizing** server state in React
  - Replaces manual **useEffect** + **useState** patterns for data fetching
  - Handles **loading**, **error**, and **stale states** automatically
- Core hooks:
  - **useQuery()** — fetch and cache data from a server
  - **useMutation()** — send POST / PUT / DELETE requests
- Setup:
  - **npm install @tanstack/react-query**

TanStack Query – Example

SoftUni AI

```
import { useQuery } from '@tanstack/react-query'

function Users() {
  const { data, isLoading, error } = useQuery({
    queryKey: ['users'],
    queryFn: () => fetch('/api/users').then(res => res.json())
  });

  if (isLoading) return "<p>Loading…</p>";

  if (error) return "<p>Error!</p>";

  return data.map(user => <div key={user.id}>{user.name}</div>);
}
```

TanStack Query – Live Demo

SoftUni AI

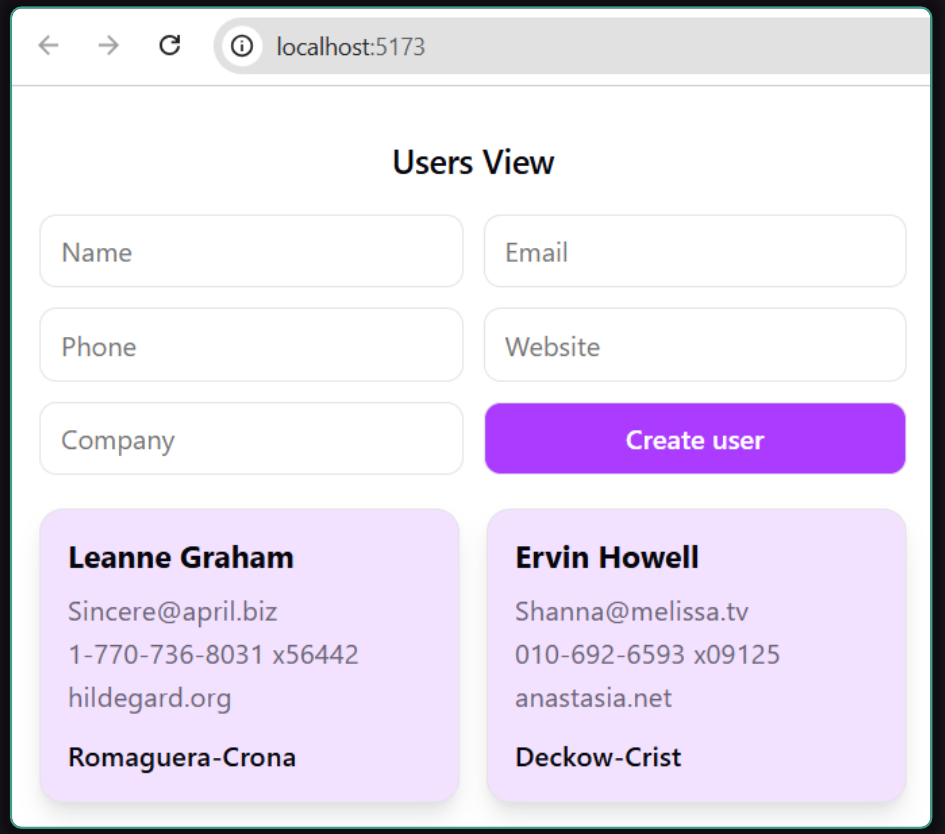
- **AI prompt** to access server API with TanStack Query:

Create `UsersView` component:

- Fetch users from https://jsonplaceholder.typicode.com/users and display them as cards

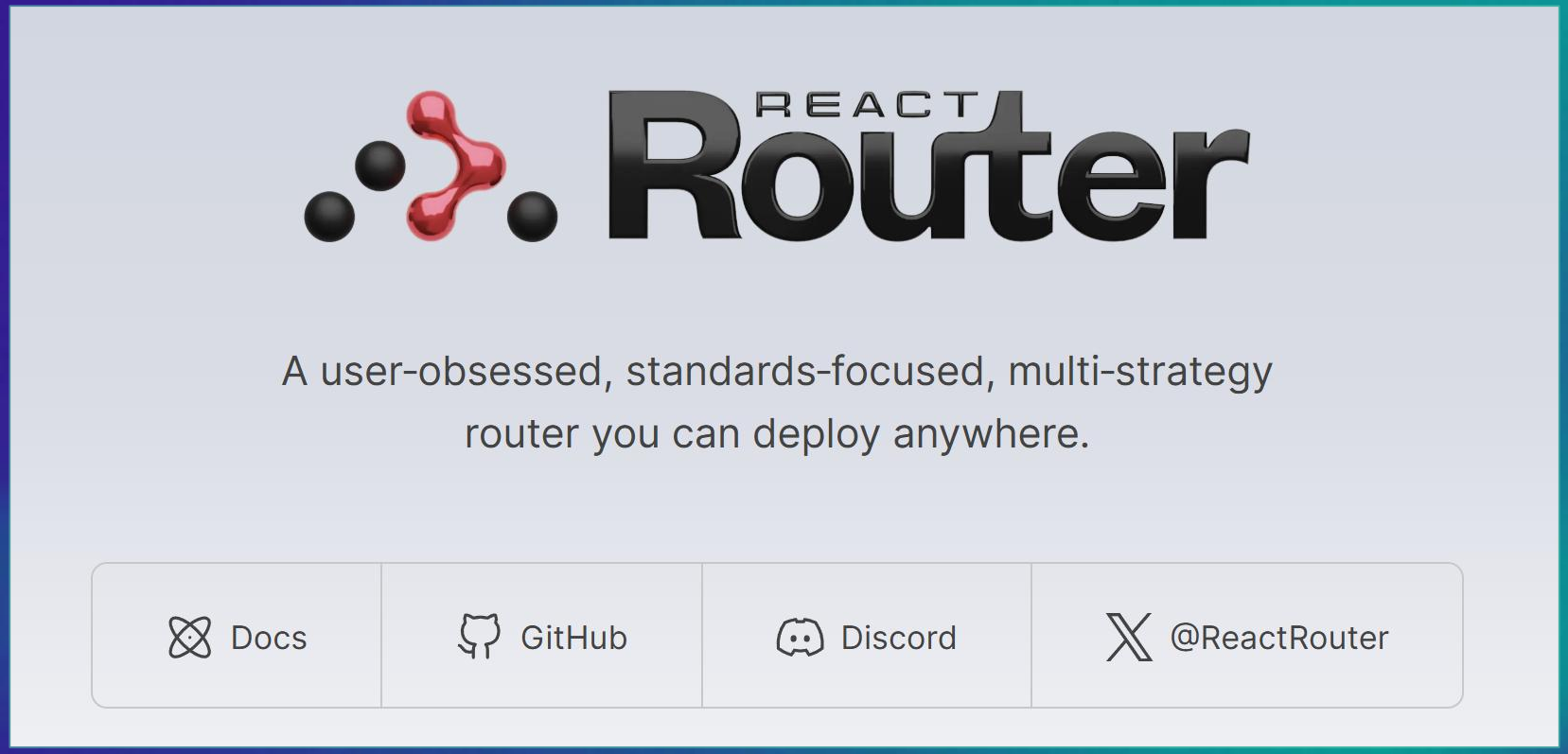
- Add a create-user form (POST), and append the created user to the list

- Use **TanStack Query** to handle API requests



React Router Basics

Client-Side Routing and Multi-Page React Apps



Intro to React Router

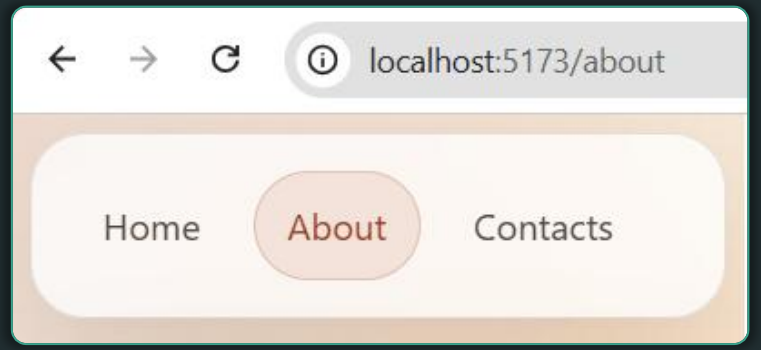
SoftUni AI

- Real apps have **multiple views** / **pages** without full reload
- **React Router** implements pages and routing in React
  - Install: **npm install react-router-dom**
  - Wrap the app with **<BrowserRouter>**
  - Define routes with **<Routes>** and **<Route>**
  - Each route renders a **different component**
- Navigation components:
  - **<Link>** instead of **<a>** → no page reload
  - **<NavLink>** adds active styling to the current page link

React Router – Example

SoftUni AI

```
import { Routes, Route } from "react-router-dom";
import { Home, About, Contacts } from "./";

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="/contacts" element={<Contacts />} />
      </Routes>
    </BrowserRouter>
  );
}
```



Navigating Between Views

SoftUni AI

- Use **<Link to="/page">** instead of **<a href="/page">**
  - Navigation happens **without page reload** — keeps the app fast

```
<Link to="/">Home</Link> |
<Link to="/about">About</Link> |
<Link to="/contacts">Contacts</Link>
```

- Use **<NavLink>** to highlights the currently active link

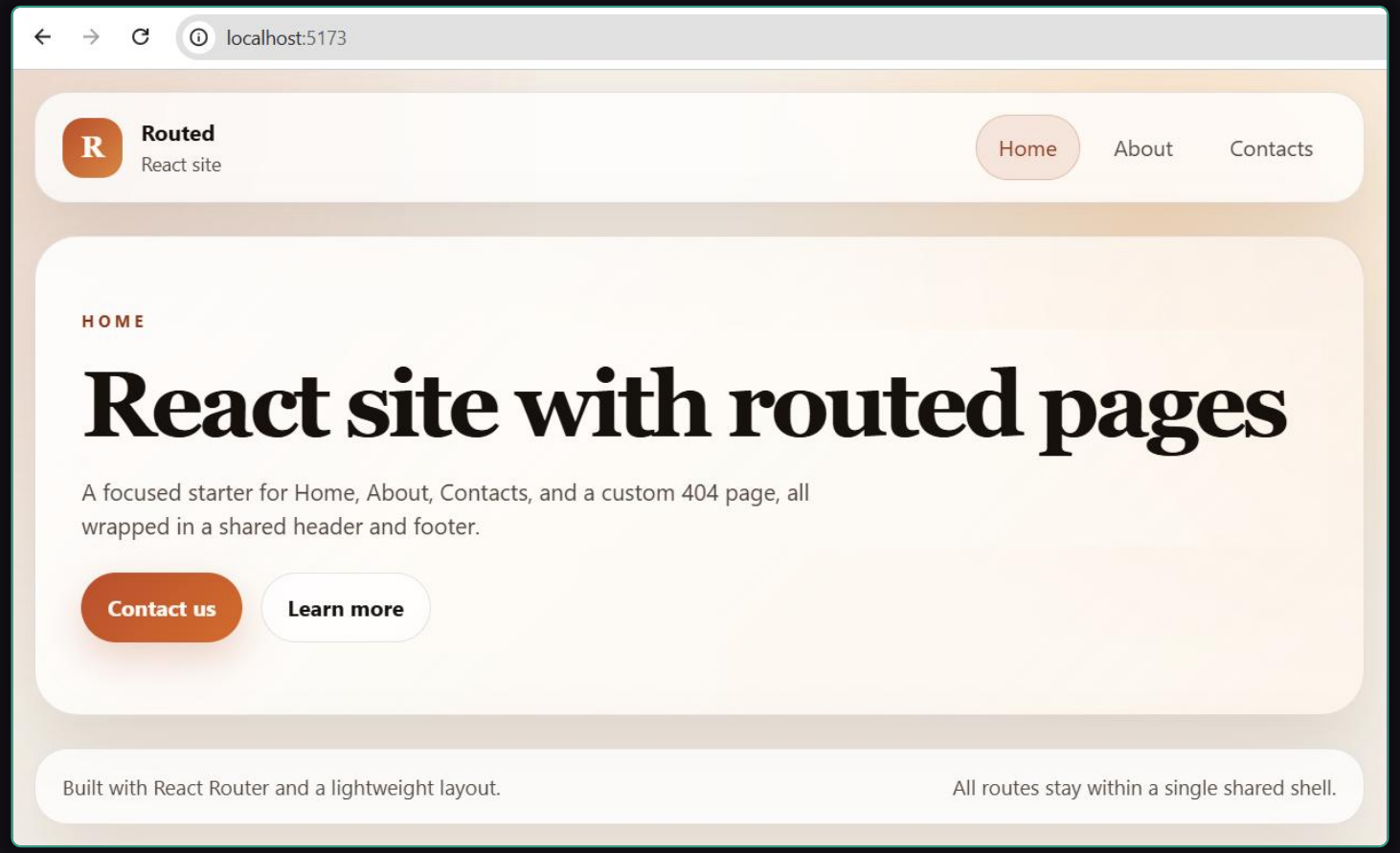
```
<NavLink to="/" className={({ isActive }) => isActive ?
"active" : ""}>Home</NavLink>
```

Example: Multi-Page React App

SoftUni AI

- AI prompt to build **multi-page React app**:

```
Create a React multi-page app with React Router: Home, About, Contacts, header, footer, NavLinks, and a 404 page
```

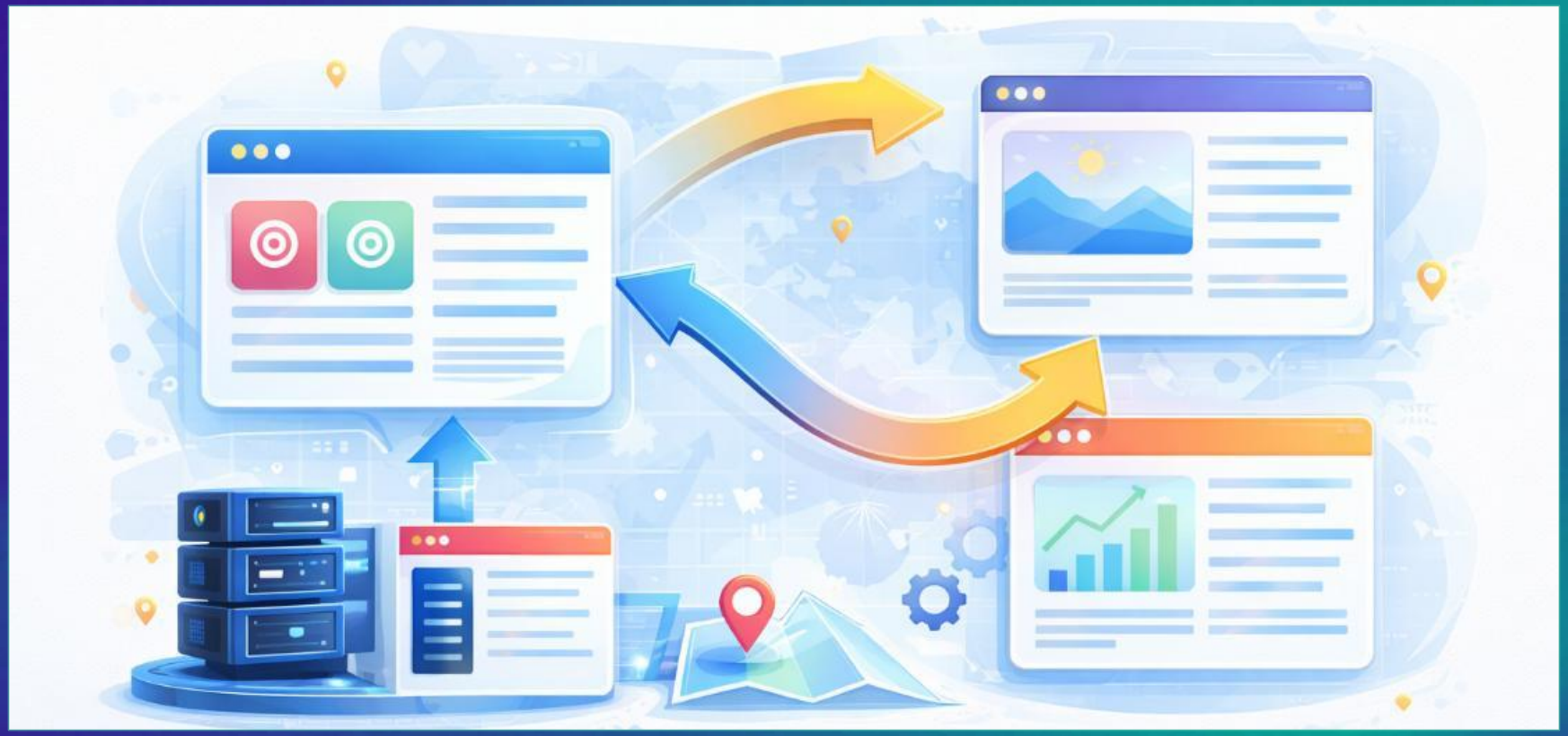


Example: Multi-Page React App

- Pages and routes:
  - **/** → Home — welcome message and intro text
  - **/about** → About — team or company description
  - **/contacts** → Contacts — form or contact details
- Shared layout features:
  - **Header** with **<NavLink>** navigation and active-link highlighting
  - **Footer** visible on all pages — site branding or copyright
  - **NotFound** page (route *) catches all unknown URLs

Dynamic Routing and Layouts

Handling Routes with Parameters



Setting Up Dynamic URL Parameters

SoftUni AI

- Some routes may contain **variables**, e. g. **/tasks/:id**
  - Read URL parameters with **useParams()**
  - Use the **id** to find and display a specific item
  - Common pattern in **details pages** (tasks, posts, products)

```
<Route path="/tasks/:id" element={<Task />} />
```

```
function Task() {
  const { id } = useParams();
  return <h1>Task ID: {id}</h1>;
}
```

Example: Task List Multi-Page App

SoftUni AI

Build a "**Task List**" multi-page app: list / view / create / edits tasks

- Use React, TypeScript, Vite and React Router
- Keep app data in the browser local storage
  - Tasks have title, deadline, owner, state, % done
  - Initially, insert sample data (a few tasks for testing)

Sample task list:

Sign Internet contract	27-Nov-2026	Maria	Signed. Internet comes in 7 days	100%
Install the WiFi router	2-Dec-2026	Steve	Not started	0%
Install the printer	3-Dec-2026	Steve	Printer prepared, waiting for the router	50%

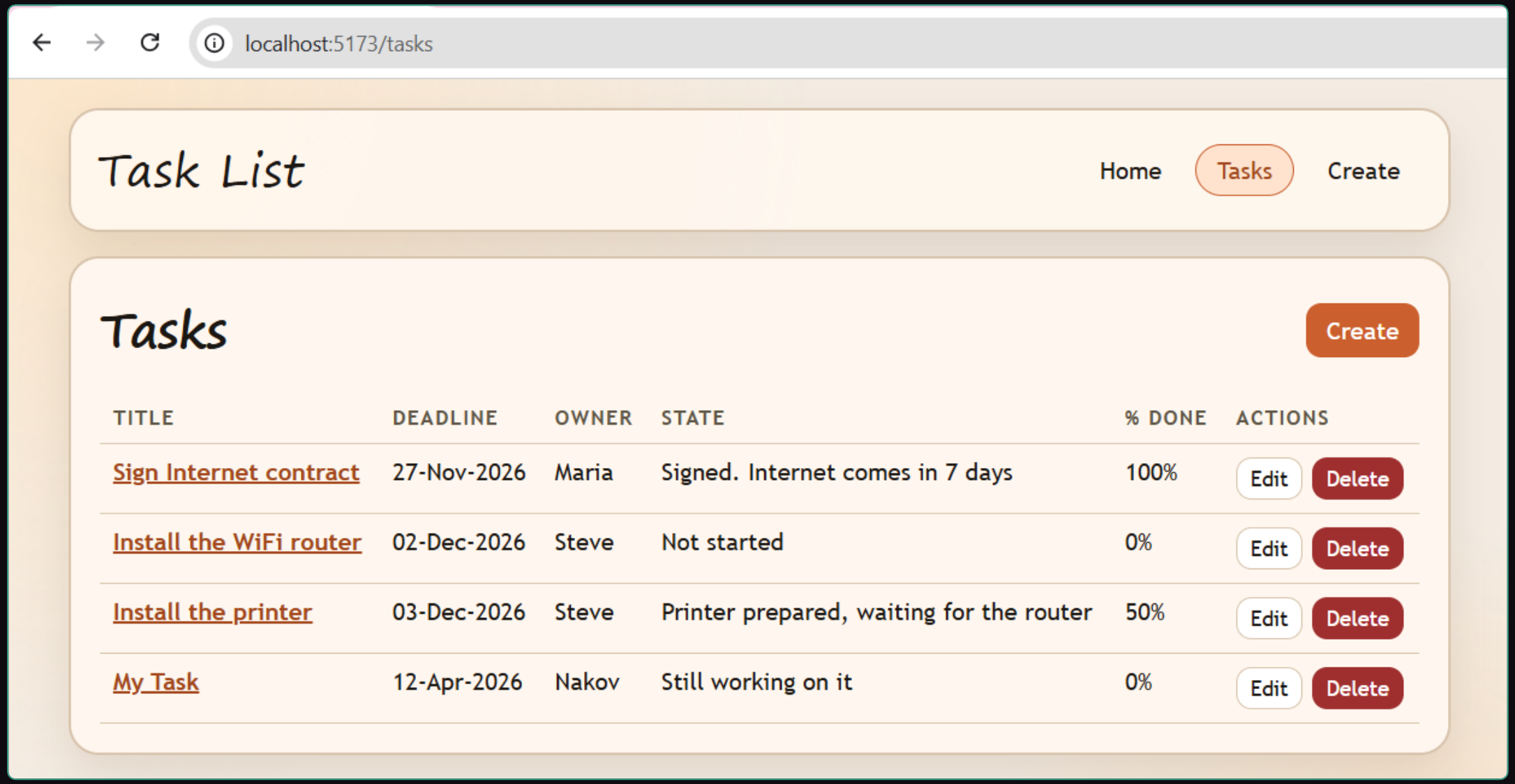
Example: Task List Multi-Page App

SoftUni AI

- Implement separate pages for each action (use React Router):
  - `/` - Home page -> shows a welcome screen + links
  - `/tasks` - Tasks list page -> list tasks in a table with [Create] / [Edit] / [Delete] buttons
  - `/tasks/:id` - Show task details, with [Edit] / [Delete] buttons
  - `/tasks/new` - Create a task
  - `/tasks/:id/edit` - Edit a task
  - `/tasks/:id/delete` - Delete a task (with confirmation)

Task List Multi-Page App

SoftUni AI



Supabase with React

Database, API and Authentication for React Apps

Setting Up a Project and MCP Server

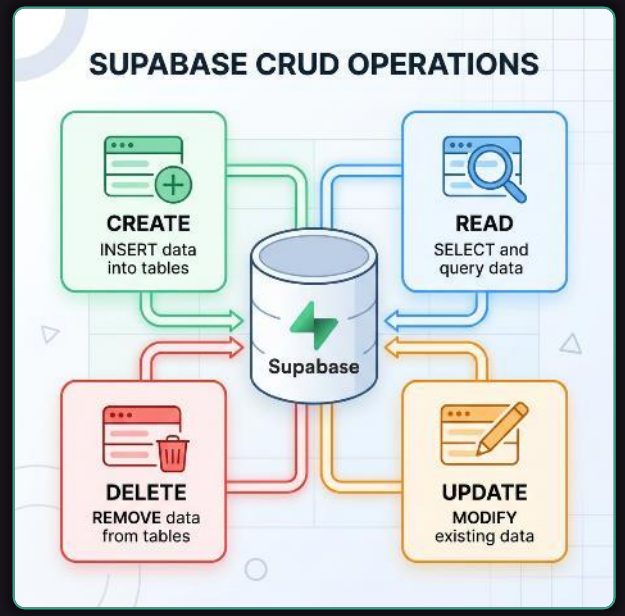
SoftUni AI

- **Supabase** is a backend platform for modern web apps
  - Provides: **PostgreSQL DB**, **auto-generated API**, **Auth**
  - Great fit for small business projects
- Supabase setup steps:
  - Create a Supabase project at **supabase.com**
  - Create **DB tables** and insert **seed data**
  - Install: **npm install @supabase/supabase-js**
  - Store URL and anon key in **.env** → never keep hardcode credentials in components

Changing Data Storage to Supabase

SoftUni AI

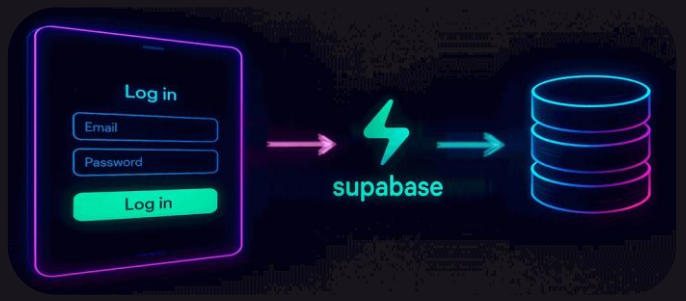
- Replace **local storage** with real backend persistence
- **CRUD operations** with the Supabase client:
  - **select()** — read rows from a table
  - **insert()** — create a new row
  - **update()** — modify an existing row
  - **delete()** — delete a row
- Row Level Security (**RLS**) in Supabase:
  - Controls who can read or modify rows — important for real apps
  - Example: everyone can **read**, owner can **edit / delete** own rows



Implementing Authentication

SoftUni AI

- Supabase Auth handles **register**, **login** and **logout**
  - **signUp()** — register a new user with email + password
  - **signInWithPassword()** — log in existing user
  - **signOut()** — log out current user
- Track user session in React apps:
  - Keep current user / session in **state or context**
  - Show **Login** / **Register** links for guests, **Logout** for signed-in users
  - Use a **ProtectedRoute** wrapper that redirects guests to login



Implementing CRUD Operations

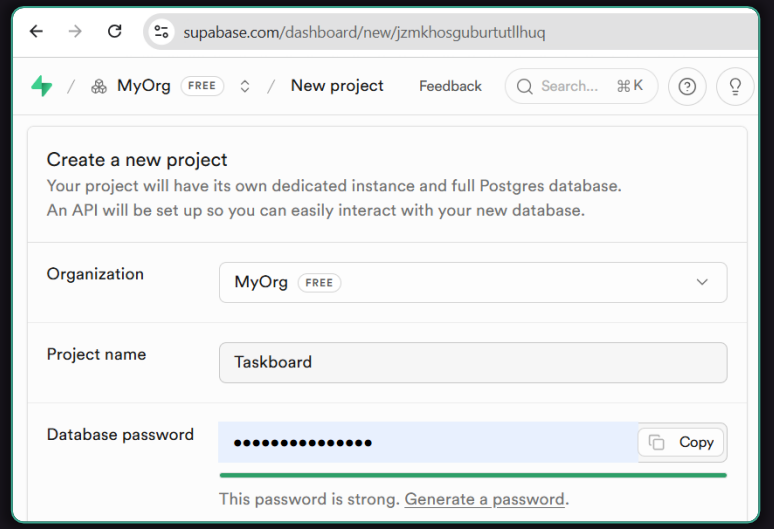
SoftUni AI

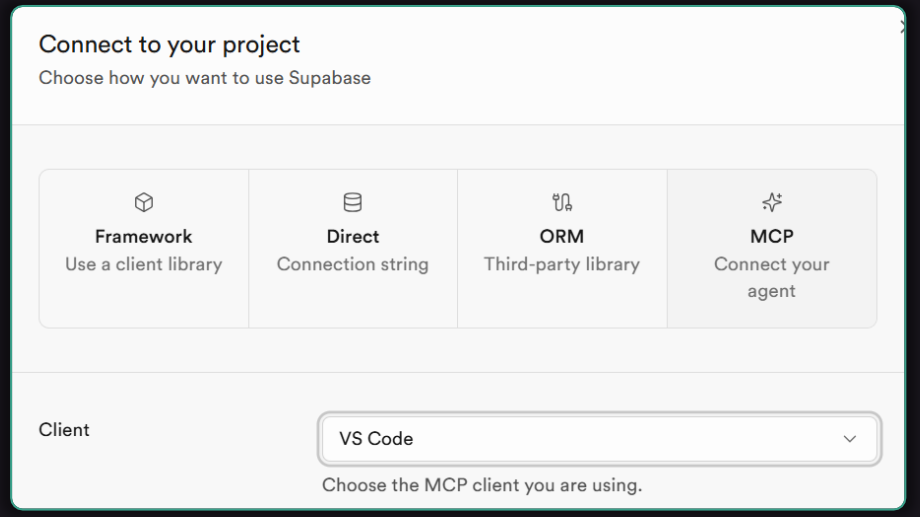
- Upgrading from **local storage** to Supabase:
  - Keep the **same UI and routes** — only swap the data layer
  - Load tasks with **supabase.from('tasks').select()**
  - Create / edit / delete through Supabase API calls
- DB tables: **auth.users**, **public.tasks**
- Tasks table and ownership:
  - Fields: **id, title, deadline, owner_id, status, percent_done**
  - RLS: signed-in users can manage **only their own** tasks

"Task List" App with Supabase

SoftUni AI

- Steps to integrate Supabase:
  - **Step 1**: Create a **Supabase project**
    - Supabase Dashboard
      → Projects
      → New Project
  - **Step 2**: Connect **Supabase MCP** to VS Code
    - Supabase Dashboard
      → Connect → MCP
      → VS Code → Workspace





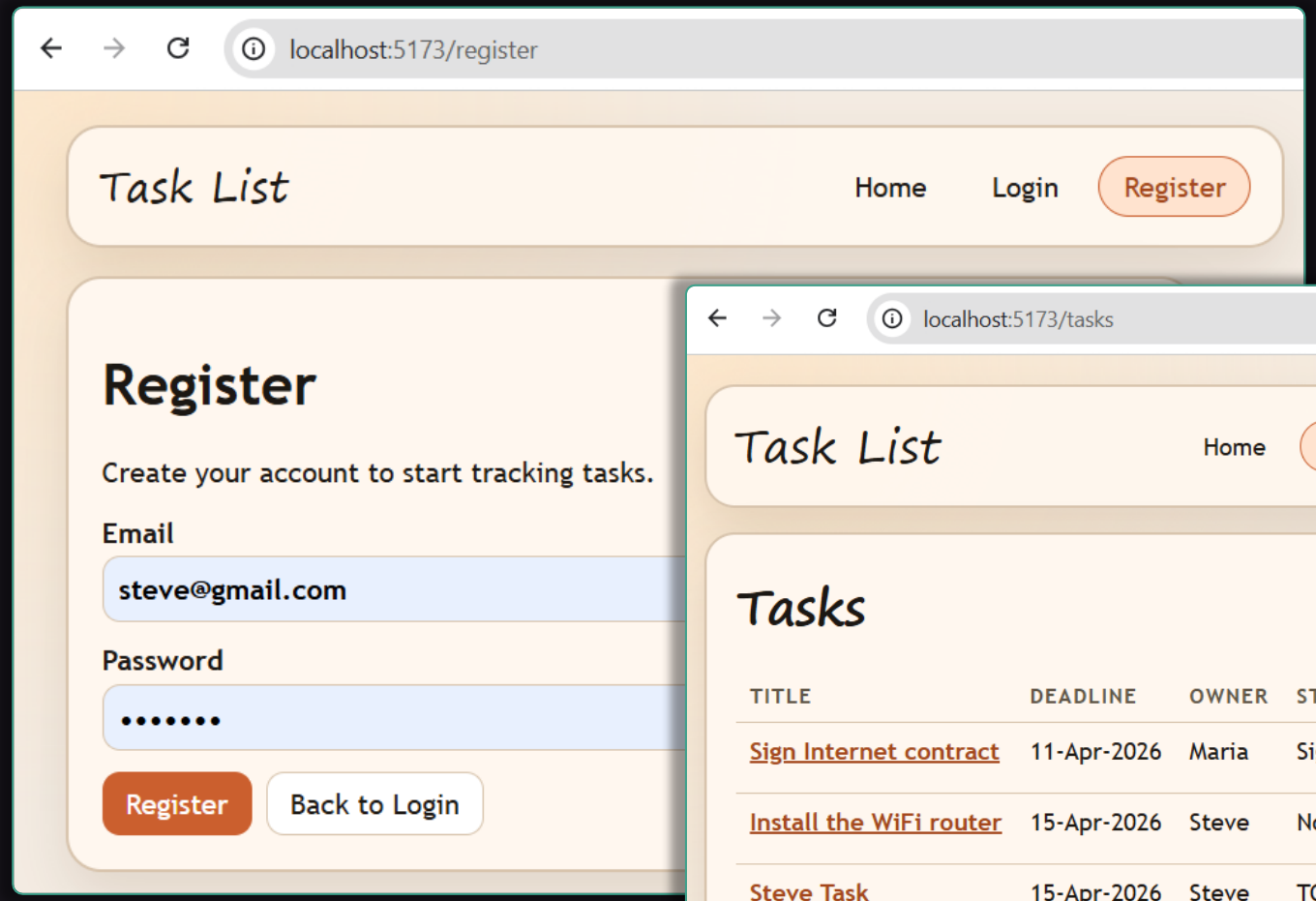
The "Task List" App with Supabase (2)

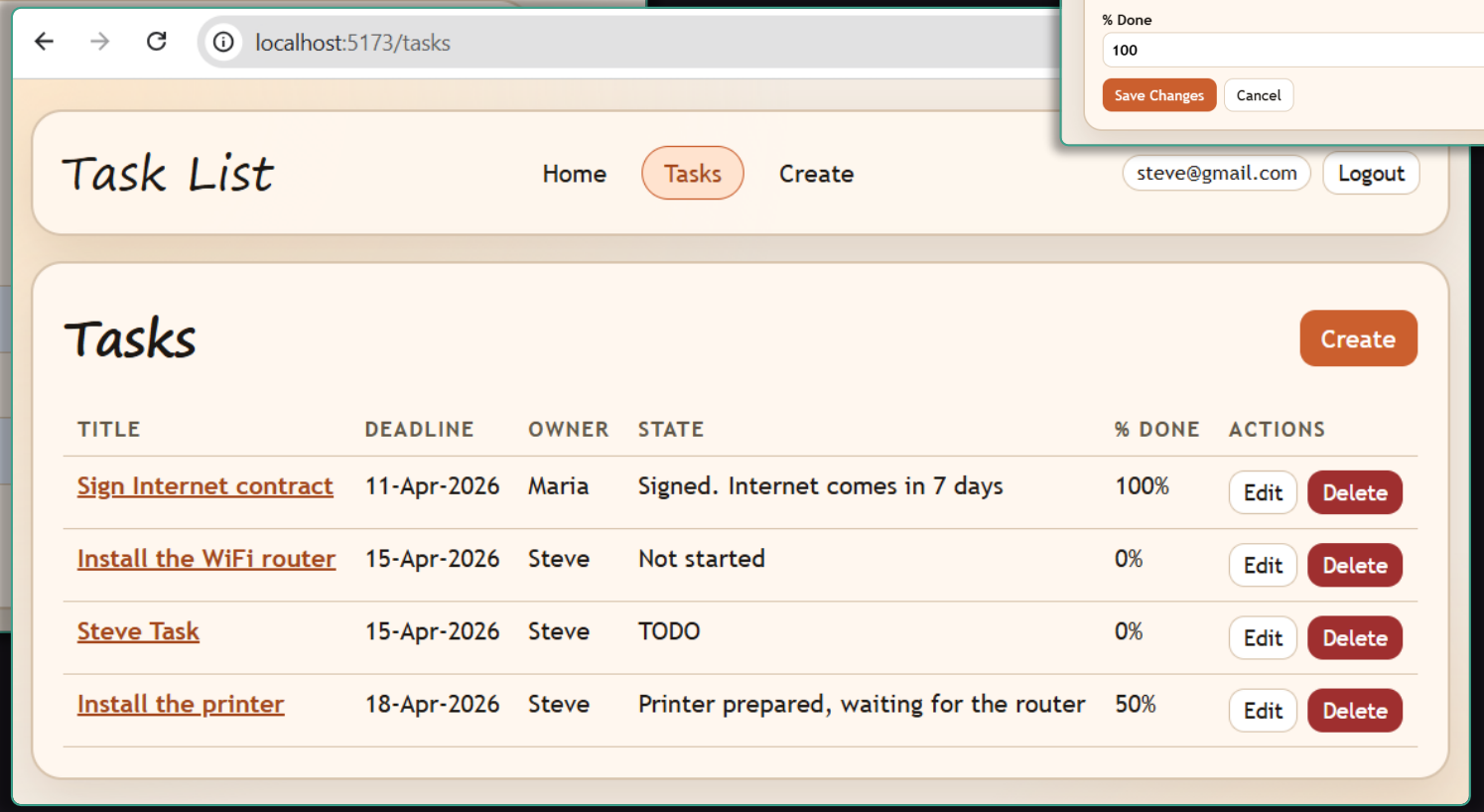
SoftUni AI

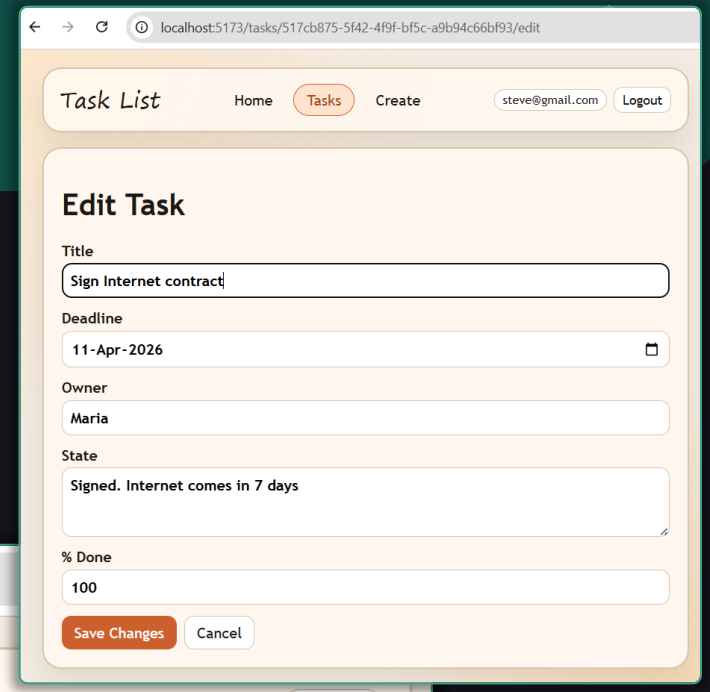
- Steps to integrate Supabase:
  - **Step 3**: Disable auth **email confirm** in Supabase
    - Supabase Dashboard → Authentication → Sign In
  - **Step 4**: Change the app storage layer:

Change the data storage layer to Supabase:
- Create **DB tables** in Supabase to hold app data + RLS policies
  - Initially, seed sample data in the DB
- Implement **authentication** (login, register, logout) with Supabase
- Implement the **CRUD operations** with Supabase API

Task List App: Screenshots







Tailwind: Modern UI Styling

Utility-First CSS for Fast UI Building

The "utility-first" Concept

SoftUni AI

- Traditional CSS: write **custom class names** for every element
  - Leads to large stylesheets, naming conflicts, unused CSS
- **Utility-first**: compose UI from small, single-purpose classes
  - Example of utility-first styling:

```
<div class="flex items-center justify-between p-4 bg-white rounded-lg shadow">Text content</div>
```

  - No custom class names needed — classes are reused across the entire app
  - Fast for **prototyping** and **consistent** styling across a team

Intro to Tailwind CSS

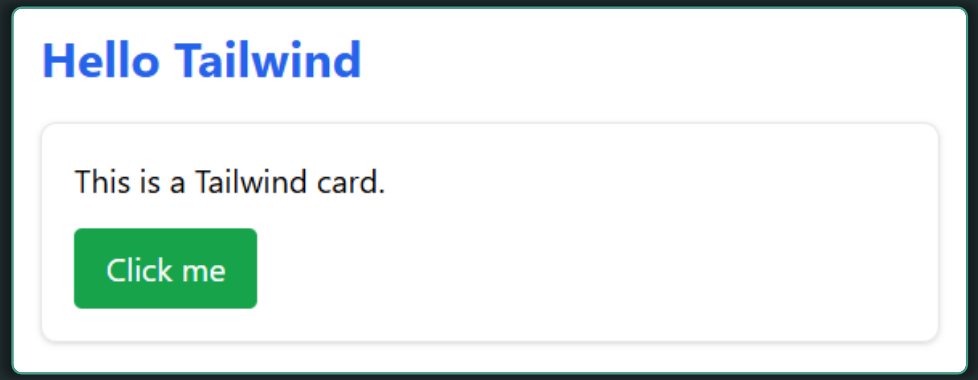
SoftUni AI

- **Tailwind CSS** is a utility-first CSS framework
  - Instead of using custom CSS styles, use **Tailwind the styles**
  - Style elements directly in your HTML using small, reusable (utility) classes (like **flex**, **text-xl**, **bg-blue-500**)
  - Responsive design via prefixes — **sm:**, **md:**, **lg:**
    apply styles at specific screen sizes (breakpoints)
  - State variants — **hover:**, **focus:**, **active:** for interactive styles without writing custom CSS
  - **Auto purge CSS**: unused Tailwind styles are removed in production builds, keeping bundle size small

Tailwind – Example

SoftUni AI

```
<script src="https://cdn.tailwindcss.com"></script>

<div class="max-w-md mx-auto mt-8">
  <h1 class="text-blue-600 text-2xl font-bold mb-4">
    Hello Tailwind
  </h1>

  <div class="border rounded-lg p-4 shadow">
    <p class="mb-3">This is a Tailwind card.</p>
    <button class="bg-green-600 text-white px-4 py-2 rounded">
      Click me
    </button>
  </div>
</div>
```



Common Utilities in Tailwind

SoftUni AI

- **Layout & flex**: flex, grid, items-center, justify-between, gap-4
- **Spacing (padding / margin)**: p-4, px-6, py-2, m-2, mt-4, mb-8
- **Size**: w-full, max-w-4xl, h-48, min-h-screen
- **Typography**: text-xl, font-bold, text-gray-700, leading-relaxed
- **Visual**: bg-white, border, rounded-lg, shadow-md
- **Buttons**: bg-blue-600 text-white px-4 py-2 rounded hover:bg-blue-700

Hover States and Transitions

- **Hover** effects: prefix to apply styles on mouse over
  - Example: **hover:bg-blue-700 hover:scale-105 hover:shadow-lg**
- Smooth transitions with **transition** utilities:
  - **transition duration-300 ease-in-out**
  - Controls which CSS property is animated — color, transform, shadow, all
- Example: interactive card
  - **bg-white rounded-xl shadow-md hover:shadow-xl hover:scale-105 transition duration-300**

Tailwind Landing Page – Example

SoftUni AI

- AI prompt to build a modern **landing page** with Tailwind:

Build a modern **landing page** with **HTML** and **Tailwind** (no custom CSS):

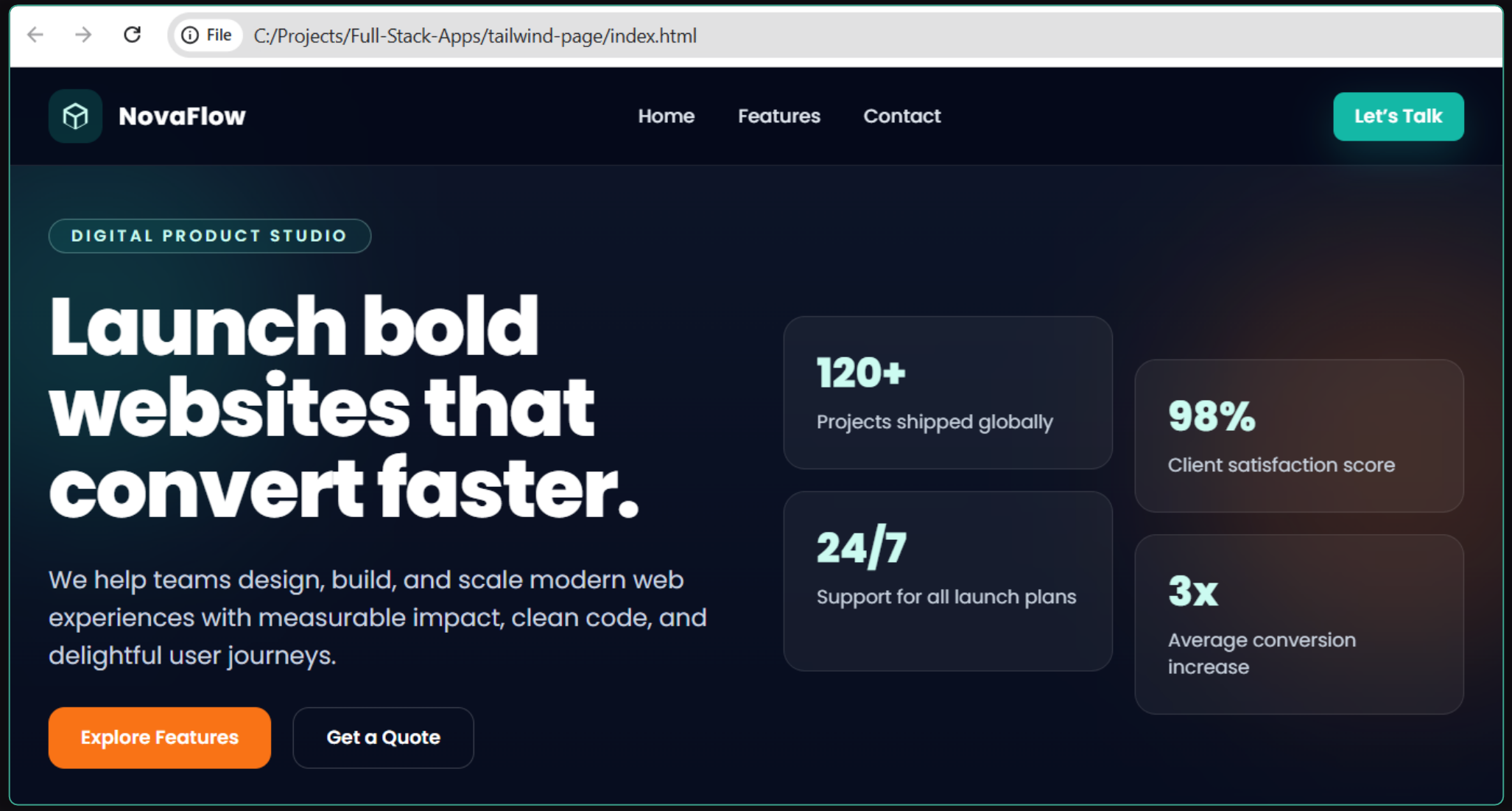
- **Header** — logo, nav links, sticky on scroll
- **Hero section** — headline, subtitle, CTA button
- **Features section** — 3 responsive feature cards with icons
- **Contacts section** — address, map, email, simple contact form
- **Footer** — copyright, links, social icons

Implement a **fully responsive layout** for mobile, tablet and desktop

Implement hover **effects**, **transitions** and **animations** for better UI

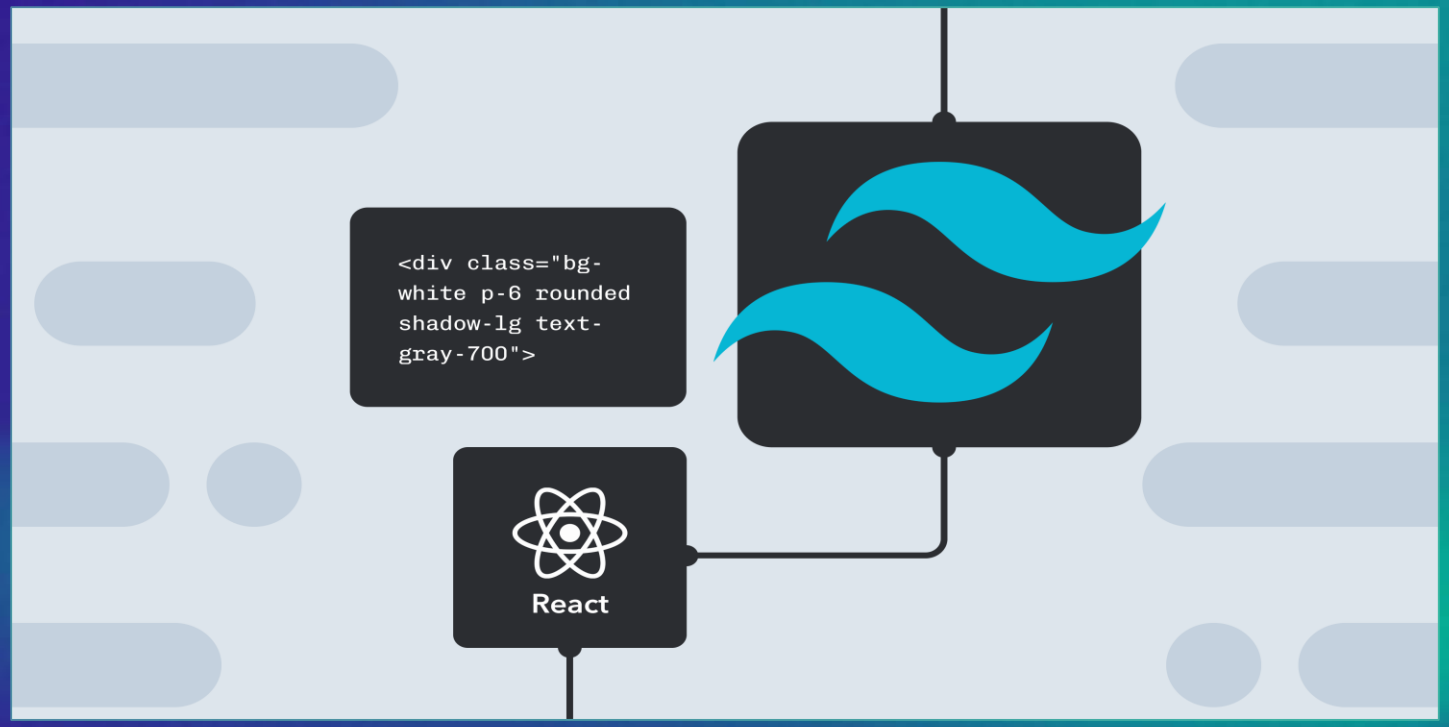
Tailwind Landing Page – Example (2)

SoftUni AI



React + Tailwind

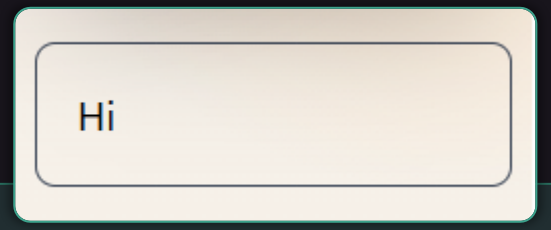
Project Setup from Scratch:
React + TypeScript + Tailwind + React Router



Installing and Using Tailwind

SoftUni AI

- Installing Tailwind in a Vite project:
  1. **npm install -D tailwindcss @tailwindcss/vite**
  2. Add the Vite plugin for Tailwind to **vite.config.js**: **plugins: [react(), tailwindcss()]**
  3. Add **@tailwind** directives to global **index.css** file: **@import "tailwindcss";**
- Using Tailwind classes in React components:

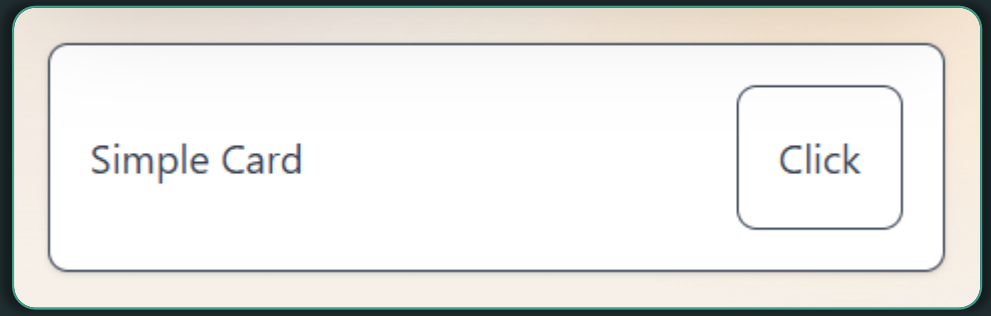



```
export default function TailwindExample() {
  return <div className="border border-gray-700 rounded-lg p-4">Hi</div>
}
```

React with Tailwind CSS – Example

SoftUni AI

```
export default function TailwindPage() {
  return (
    <div className="p-4 text-gray-700">
      <div className="bg-white border rounded-lg shadow p-4
flex items-center justify-between">
        <span>Simple Card</span>
        <button className="border rounded-lg p-4 hover:bg-gray-
200">Click</button>
      </div>
    </div>
  );
}
```



Bootstrap vs. Tailwind

SoftUni AI

- **Bootstrap** — component-based CSS framework:
  - Very powerful, fast start, UI component — fast start
  - Pre-built components: **btn, card, modal, navbar**
  - Harder to customize — often looks "bootstrap-y"

- **Tailwind** — utility-first CSS framework:

  - Build anything from scratch — no default component look
  - Highly customizable via **tailwind.config.js**
  - Works perfectly with the **React component architecture**
  - Purges unused styles in production — very small CSS bundle

Tailwind and AI

SoftUni AI

- AI tools work great with **Tailwind**
  - Classes are descriptive and predictable
  - Ask the AI to "***style with Tailwind***" — output is immediately usable
  - No need of external CSS — AI generates inline classes
  - **Easier to maintain** — changing the styles affects only a single file
- Responsive and state variants:
  - Responsive: **sm:, md:, lg:** — e. g. **md:grid-cols-3**
  - Hover / focus: **hover:bg-blue-600**, **focus:ring-2**
  - Dark mode: **dark:bg-gray-900**, **dark:text-white**

Why React + Tailwind?

SoftUni AI

- React and Tailwind are a **natural fit**:
  - Both work at the component level — styles live alongside TSX
  - No need for separate CSS modules or styled-components
  - AI generates great **React** + **Tailwind** — very productive workflow
  - Used by many real-world production apps and design systems
- Tech stack for modern front-end projects:
  - **React** + **Vite** (build tool) + **TypeScript** + **Tailwind** + **React Router**
  - **Supabase** as a backend (DB + Auth + Storage)

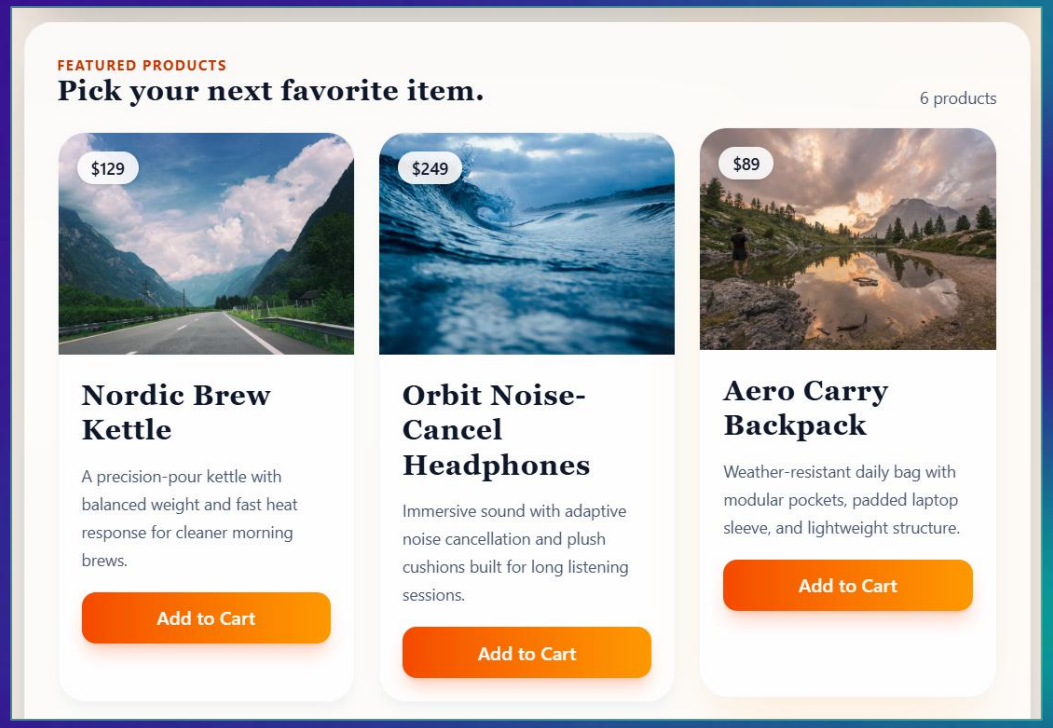
Initializing a Modern React Project

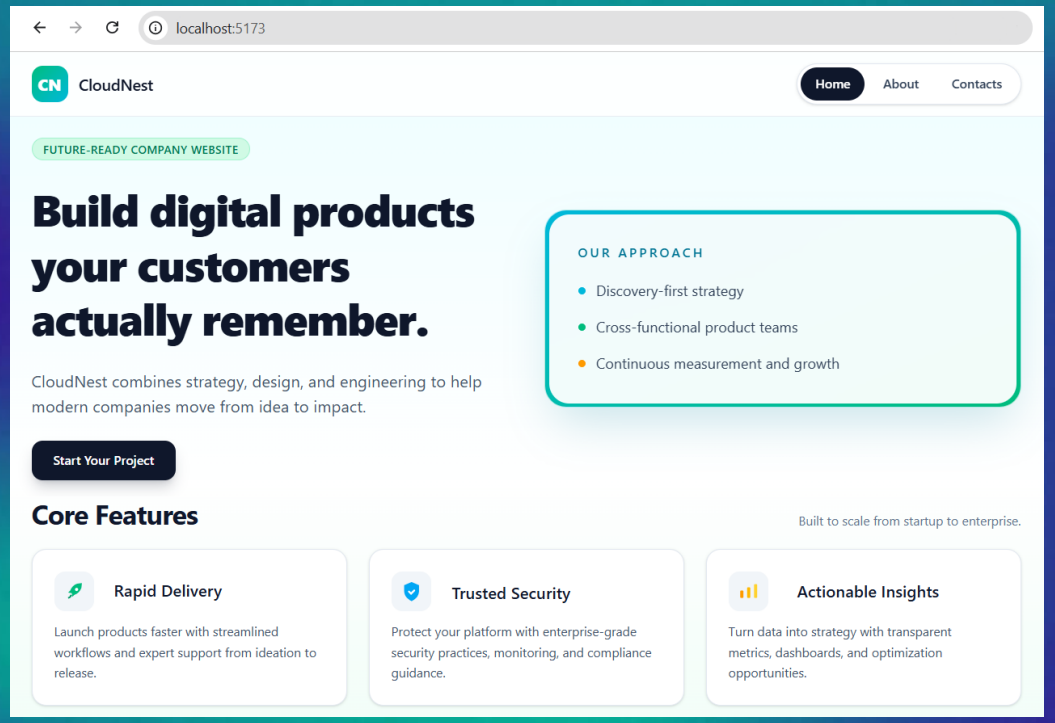
SoftUni AI

- Create **React** app with **Vite** + **TypeScript** + **Tailwind**:
  - **npm create vite@latest . -- --template react-ts --no-interactive**
  - **npm install react-router-dom**
  - **npm install -D tailwindcss @tailwindcss/vite**
- Recommended project structure:
  - **src/pages/** — one file per route page
  - **src/components/** — reusable UI components
  - **src/App.tsx** — router + layout shell
  - **src/main.tsx** — app entry point (imports global **styles.css**)

React + Tailwind Examples

Modern Product Card | Company Website

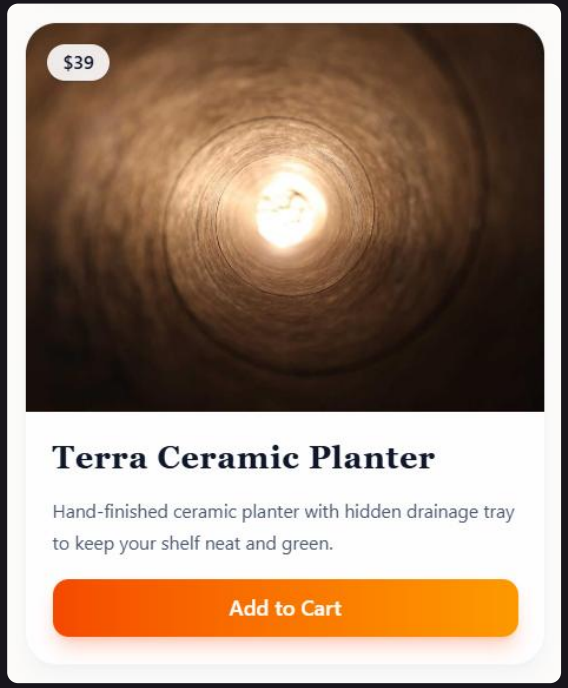




Creating a Product Card Component

SoftUni AI

- Build a **ProductCard** React component with **Tailwind**
  - Product **image**, **name**, **price**, short **description**
  - **Add to cart** button with Tailwind hover state
  - Rounded corners, shadow, clean whitespace
- Render 6 cards in a responsive grid:
  - **grid grid-cols-1 sm:grid-cols-2 md:grid-cols-3 gap-6**
  - 1 column on mobile, 2 columns on tables, 3 columns on laptop

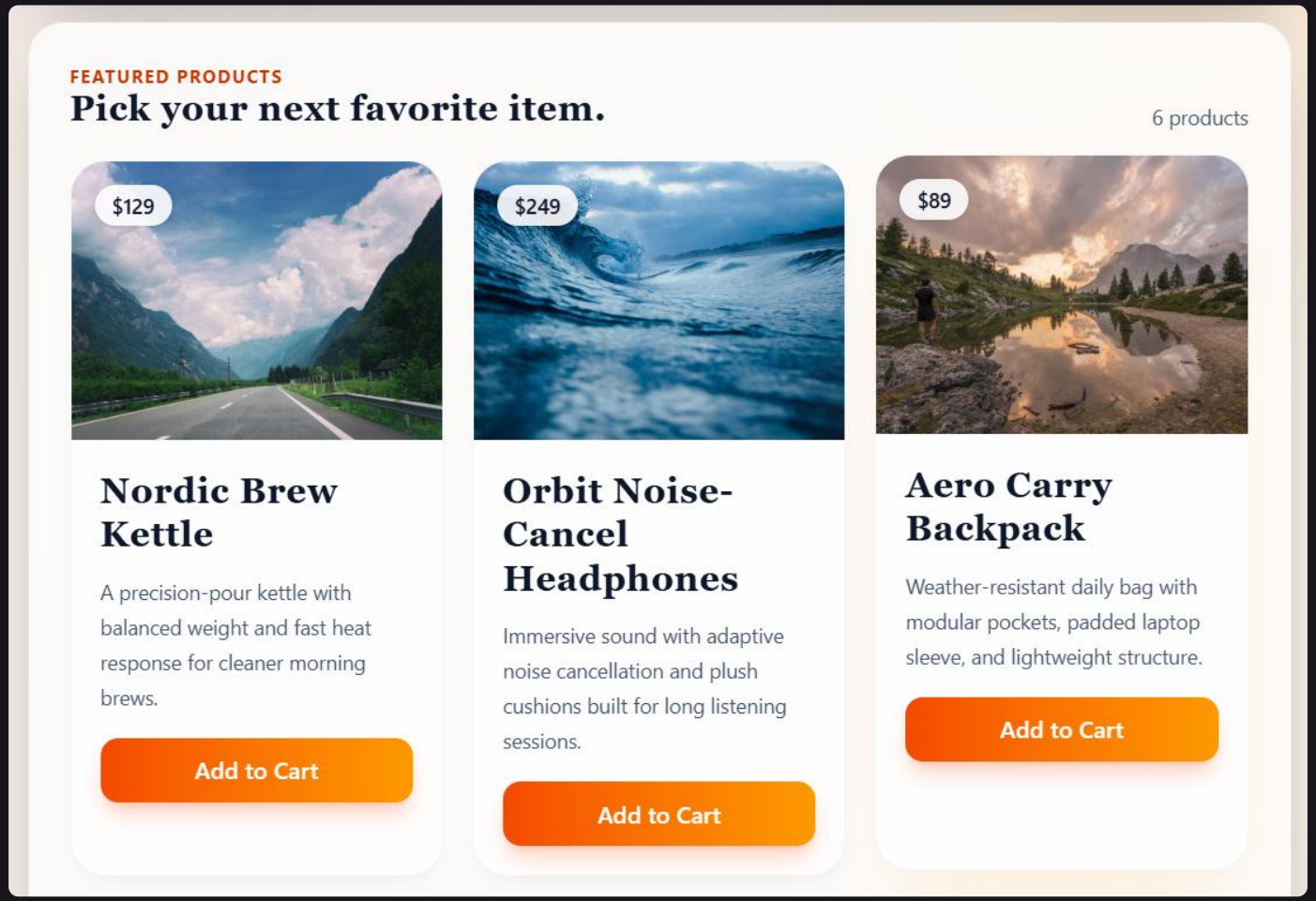


Creating a Product Card Component

SoftUni AI

- AI prompt to build a **ProductCard** with Tailwind:

```
Create a React ProductCard component using Tailwind CSS. Render a responsive grid of 6 product cards with image, name, price, description and a styled button.
```



Adding Hover Effects

SoftUni AI

- Enhance the product card with **hover effects:**
  - Card lifts on hover: **hover:shadow-xl hover:-translate-y-1**
  - Button changes color: **hover:bg-blue-700 active:scale-95**
  - Always pair with **transition duration-300** for smooth animation
- Image zoom effect:
  - Wrap image in **overflow-hidden** container, add **hover:scale-110 transition** to the image

Example: Company Website

SoftUni AI

- Build a company website component with **Tailwind**
  - **Header** — logo, nav links, sticky on scroll
  - **Hero section** — headline, subtitle, CTA button
  - **Features section** — 3 responsive feature cards with icons
  - **Footer** — copyright, links, social icons
  - **About page** — hero section + company mission + team cards
  - **Contacts page** — address, map, email, simple contact form
- Implement **fully responsive layout**
  (for mobile, tablet and desktop)

AI Prompt: Tailwind Company Site

SoftUni AI

- AI prompt to generate a **company website** with Tailwind:

Create a new React app with Vite, TypeScript, React Router, Tailwind (configure with **tailwindcss/vite**)

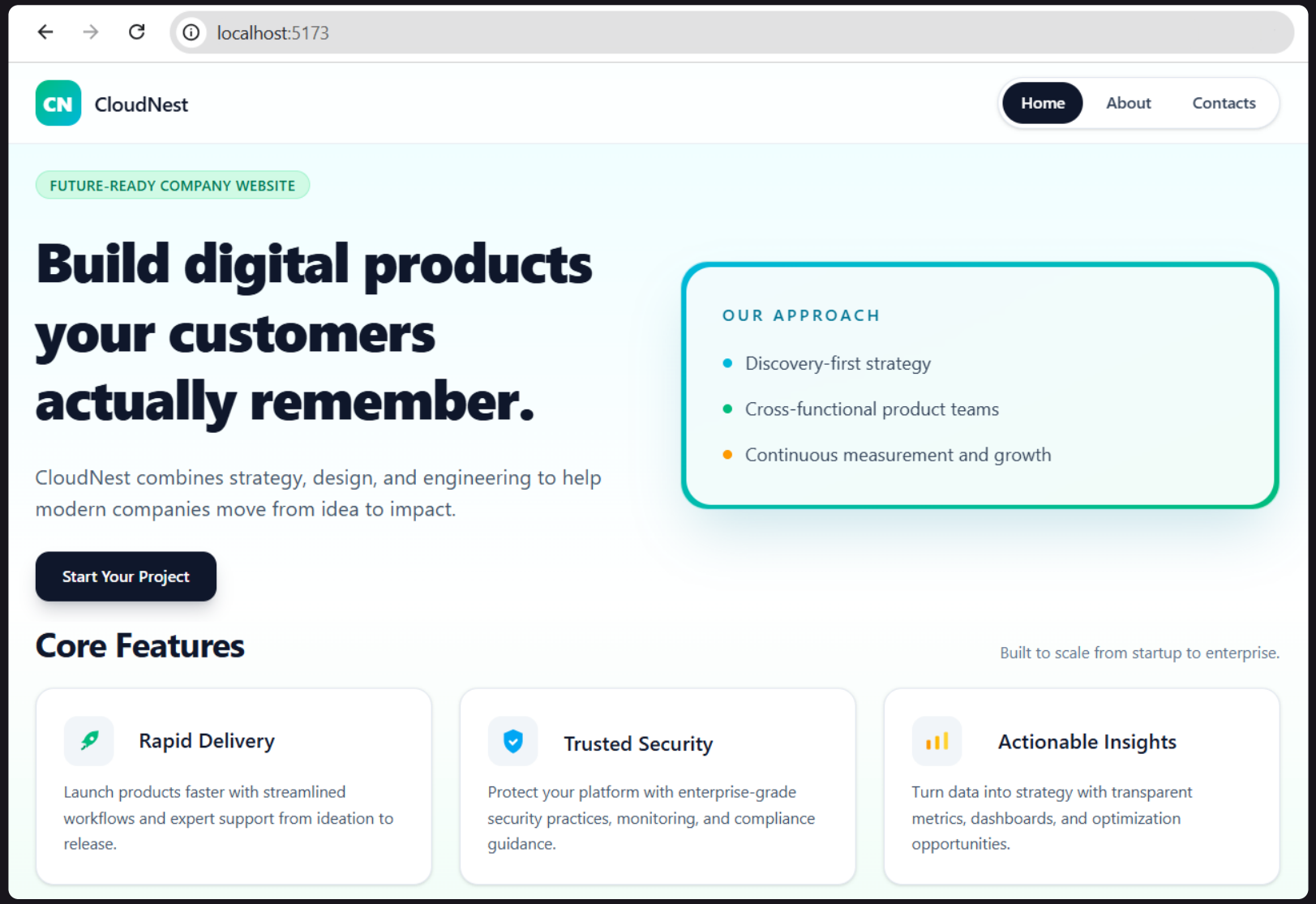
Build a company website component with **Tailwind** (no custom CSS):

- **Header** — logo, nav links, sticky on scroll
- **Hero section** — headline, subtitle, CTA button
- **Features section** — 3 responsive feature cards with icons
- **Footer** — copyright, links, social icons
- **About page** — hero section + company mission + team cards
- **Contacts page** — address, map, email, simple contact form

Implement a **fully responsive layout** for mobile, tablet and desktop

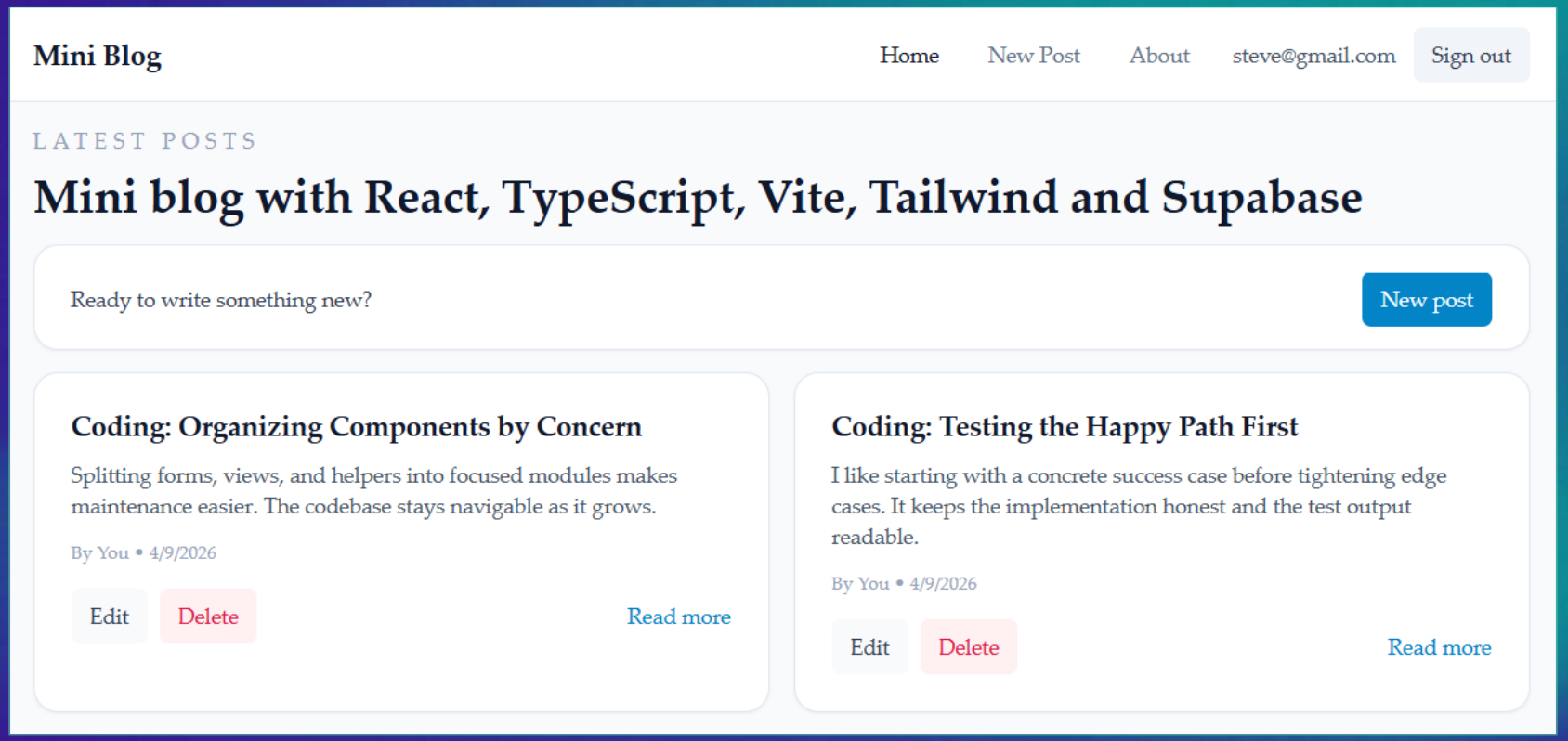
Example: Company Site with Tailwind

SoftUni AI



Exercise: Mini-Blog App

A Complete App with React + Tailwind + React Router + Supabase



Mini-Blog App

SoftUni AI

- We want to build a fully-functional **React** app with **Tailwind** + **React Router** + **Supabase**
- A simple **blog system**:
  - Visitors browse / view posts, register, login
  - Logged-in users create new posts
  - Post owners view / edit / delete their posts
- Let's build it **step by step** (with multiple prompts)

Step 1: Initialize the Project

SoftUni AI

- Create a modern front-end React project:
  - **Vite + TypeScript + React** — project scaffold
  - **Tailwind CSS** — install and configure **Tailwind for Vite plugin**
  - **React Router** — configure BrowserRouter, routes, shared layout
  - **Supabase client** — create **supabase.ts**, store API keys in **.env**
- Use the classical app folder structure:
  - **src/pages/**, **src/components/**, **src/types/**, **src/lib/supabase.ts**, **supabase/migrations/**

Step 2: App Pages

SoftUni AI

- Build blog **routes** and **pages** (initially empty):
  - **/** → Home page — list all posts (title, excerpt, author, date)
  - **/post/:id** → Single post — full content, edit/delete links if owner
  - **/login**, **/register** → User auth forms (login / register)
  - **/post/new** , **/post/:id/edit**, **/post/:id/delete** →
    Create / edit / delete posts — protected routes (auth required)
  - **/about** → About page
- Implement simple **app layout**:
  - **Header** (logo + nav links) + **page** content + **footer**

Step 3: Supabase Project + MCP

SoftUni AI

- Create a **Supabase project** for the Blog app
- Install the **Supabase MCP** in the VS Code workspace
- Create the **DB tables** (with Supabase migration, using MCP):
  - **posts (id, title, content, author_id, created_at)**
  - Link **author_id** to **auth.users** for ownership tracking
  - Apply RSL policies (Row Level Security):
    - **SELECT** — everyone (public read)
    - **INSERT** — authenticated users only
    - **UPDATE / DELETE** — only where **author_id = auth.uid()**

Step 4: App Implementation

SoftUni AI

- Implement the **app logic**:
  - Visitors browse / view posts, register, login
  - Logged-in users create new posts
  - Post owners view / edit their posts
- Split the app into simple **modular components**
- Style the components with **Tailwind**
- Implement modern **responsive UI** with fonts, icons, effects, transitions, animations

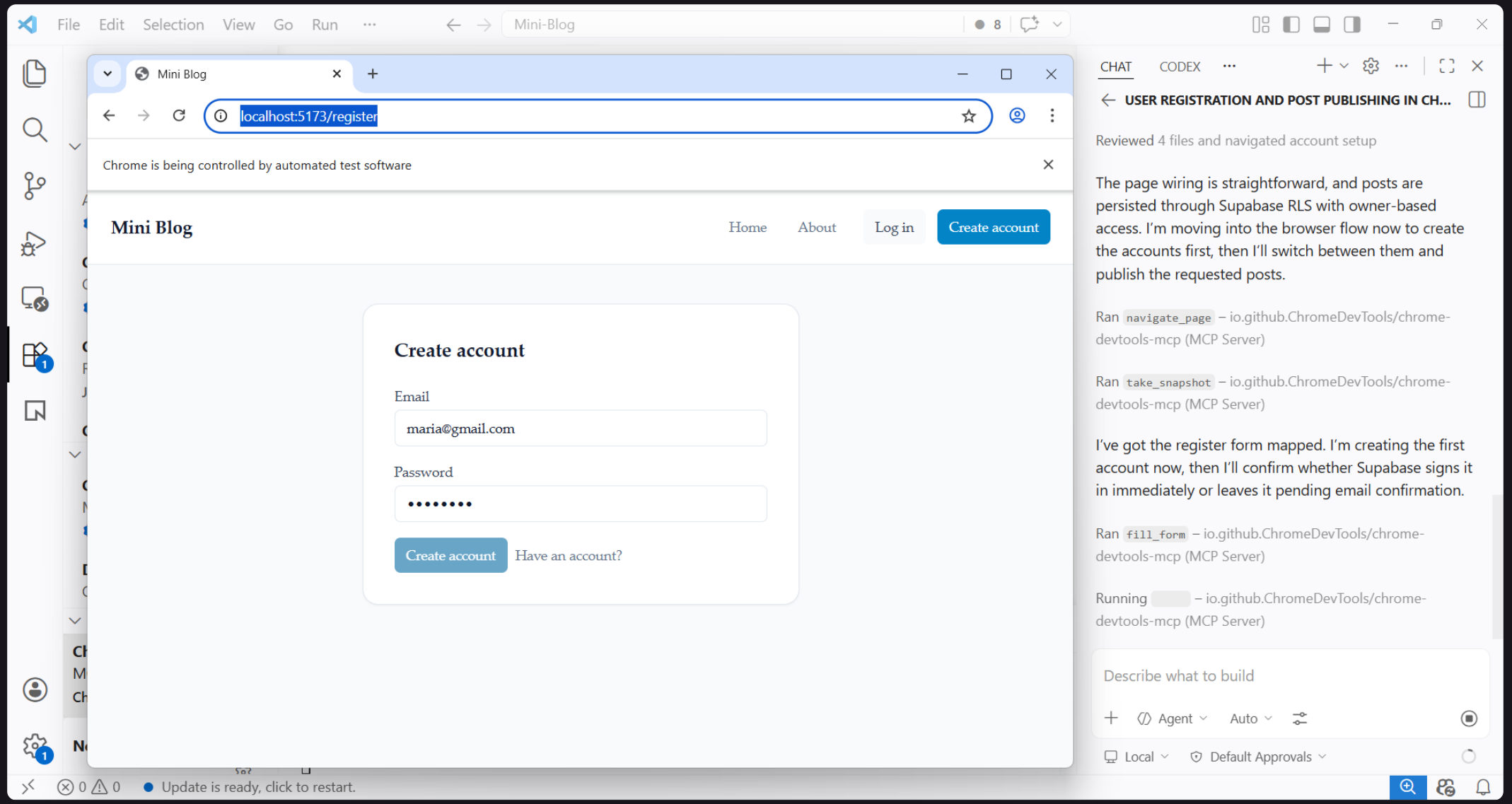
Step 5: Insert Sample Data

SoftUni AI

- Install "**Chrome DevTools MCP**" in VS Code
- Using Chrome DevTools MCP
  - **Browse** the Blog app
  - **Register** two users:
    - maria@gmail.com / maria123
    - steve@gmail.com / steve123
  - **Login** as maria@gmail.com → **publish** 3 posts about travel
  - **Login** as steve@gmail.com → **publish** 4 posts about coding

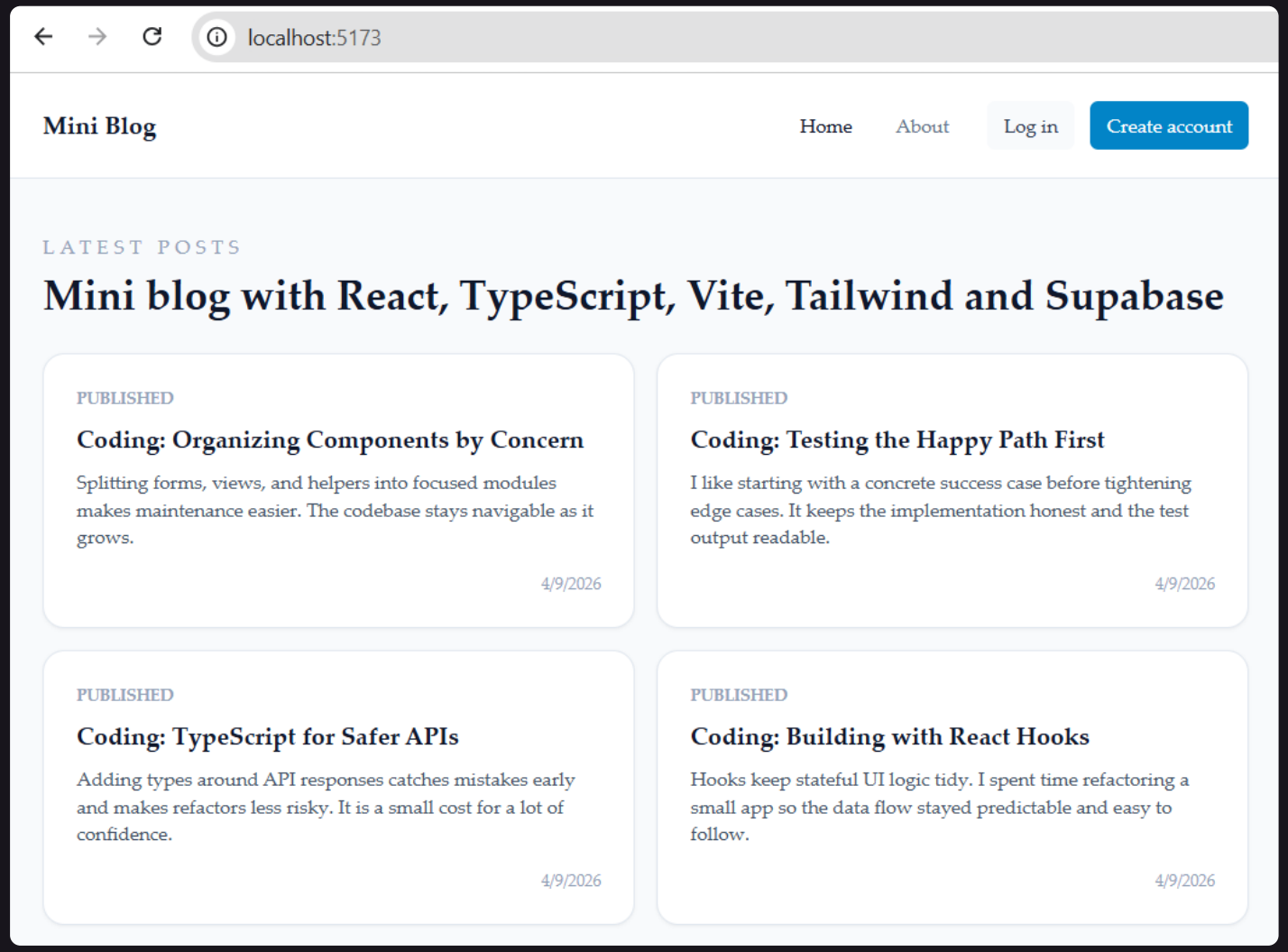
Sample Data with Chrome MCP

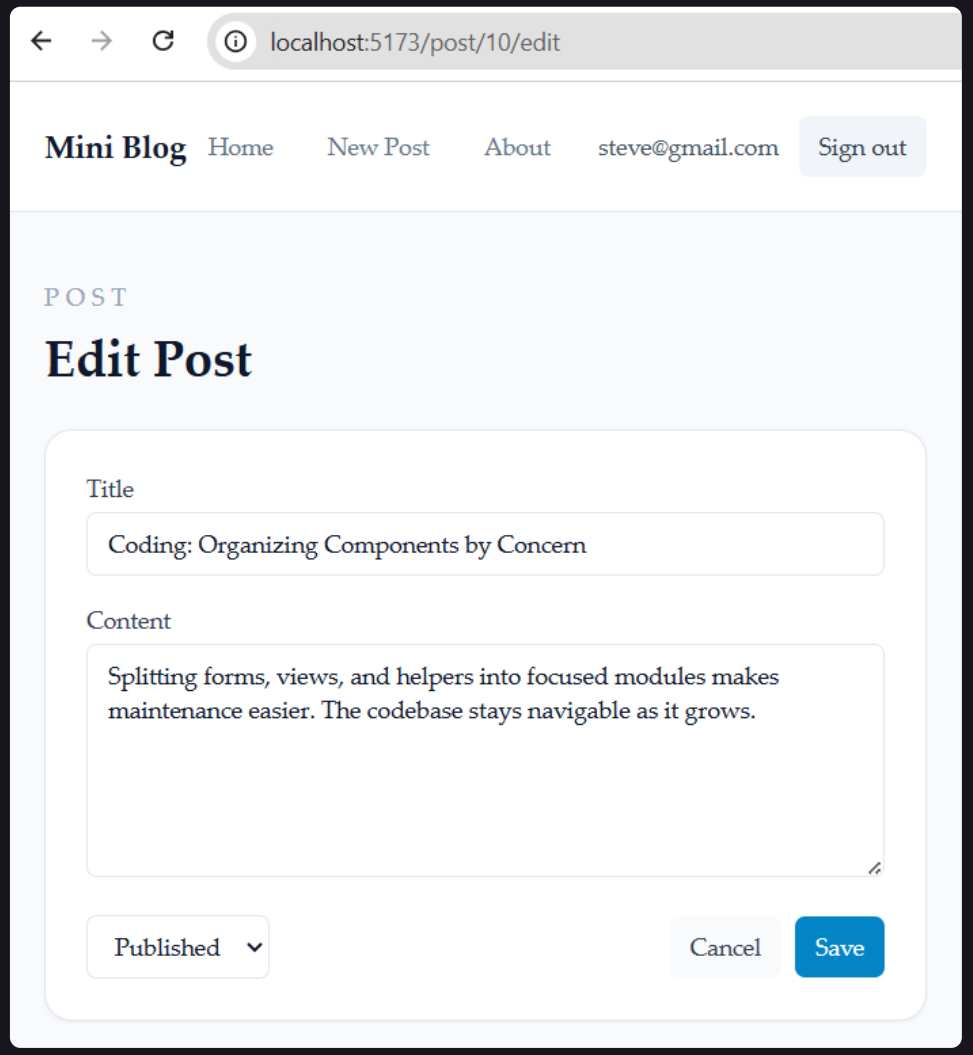
SoftUni AI



Blog App: Screenshots

SoftUni AI





Lesson Summary

SoftUni AI

- **Fetching data** with useEffect() and fetch() / TanStack Query
- **Client-side routing** with React Router — pages, layouts, links
- **Dynamic routes** with URL parameters and useParams()
- **Supabase** for backend (database + API + authentication)
- **Tailwind CSS** for fast, utility-first UI styling
- **Creating** full React apps with Vite, Tailwind, React Router and Supabase — with pages, data and auth

© SoftUni AI – https://ai.softuni.bg. Copyrighted document. Unauthorized copy, reproduction or use is not permitted.

Postbank – Exclusive Partner for SoftUni AI

- One of the leading **banking institutions** in Bulgaria
- Member of the Eurobank Group with € 103 billion assets
- Innovative trendsetter with next generation **beyond banking**, transforming today, empowering tomorrow
- Certified **Top Employer 2025** by the international Top Employers Institute
- Proven people care and **wellbeing initiatives**
- Benefits and unlimited access to professional, **personal and leadership trainings and programs**
- www.postbank.bg / careers.postbank.bg

Diamond Partners of Software University

Software University

Coca-Cola HBC Bulgaria

Postbank

top EMPLOYER България Bulgaria 2025

FOR A BETTER WORLD OF WORK

SUPER HOSTING .BG

encorp.ai

createX

DRAFT KINGS

THE CROWN IS YOURS

INDEAVR

Serving the high achievers

Diamond Partners of SoftUni Digital

SoftUni Digital

Coca-Cola HBC Bulgaria

Postbank

top EMPLOYER България Bulgaria 2025

FOR A BETTER WORLD OF WORK

SUPER HOSTING .BG

encorp.ai

createX

INDEAVR

Serving the high achievers

DRAFT KINGS

THE CROWN IS YOURS

Diamond Partners of SoftUni Digital

SoftUni Digital

HUMAN

NETPEAK
DIGITAL GROWTH PARTNER

MarmaLAB | E-комерс агенция

ETIENYANEV
Break Your *Limits*. Live Your *Brand*.

1 FOR FIT

Aleikov Studio

Diamond Partners of SoftUni Creative

SoftUni Creative

Coca-Cola HBC Bulgaria

Postbank

top EMPLOYER България Bulgaria 2025

FOR A BETTER WORLD OF WORK

SUPER HOSTING .BG

encorp.ai

createX

INDEAVR

Serving the high achievers

DRAFT KINGS

THE CROWN IS YOURS

Organization Partners of SoftUni Creative

SoftUni Creative

creative corner

Studio π

Think Big • Design Smart

THE BUCKS TOWN'S WORK

DEFFECTO